DokLink - Development Report & Interview Q&A

The real build process, the questions it invites, and every term answered

Nirmalya Mandal - SAP Labs Interview Prep

Reconstructed from the 239-commit development history

Contents

How to use this report

This is your deep-prep document for DokLink, built from the actual commit history of the project, not a summary. It has three parts:

- **Part 1 - The development story.** The real build sequence, phase by phase, as bullet points. This is *what happened*, so you can narrate it in your own words.
- **Part 2 - Questions and detailed answers.** For each phase, the questions an interviewer is likely to ask if you mention those points, each with a full answer at your level.
- **Part 3 - The bait-terms dictionary.** Every technical term you might deliberately drop, with a detailed answer and a short "say it like this" line.

How to lead the interview with it: say a term and pause. "I fixed an N+1 query in the hospital search." Stop. Let them ask. Every pause hands you a question you already own. Only drop a term you can hold the follow-up on - this document is how you make sure you can.

Honesty and NDA note: this is your own engineering, so discussing the tech and the decisions is completely fine and expected. Keep confidential specifics out of it - no real patient data, no credentials, no exact proprietary business rules. Talk architecture and reasoning, not secrets.

Part 1 - The development story

Phase 1 - Environment and scaffolding (Jun 2025)

- Set the project up on GitHub as a **monorepo**: a `client` folder for the mobile app and a `server` folder for the backend.
- Built the mobile app in **React Native with Expo**, in TypeScript.
- First screens: Starting screen, Login, About.
- Normalised asset paths so every developer on the team could run the project cleanly.

Phase 2 - Authentication backend (Jul 2025)

- Built the **Python (Django) backend**, starting from sign-up and login.
- Login with email and password.
- Upgraded password storage to **Argon2** hashing (a stronger algorithm).
- Refactored the mobile login screen into reusable **components**.
- Conditional navigation: if a valid session exists, go to Home; otherwise the Starting screen.

Phase 3 - Core booking and features (Jan 2026)

- Implemented **emergency booking** tied to hospital management.
- Integrated the **Razorpay** payment gateway with financial tracking per booking; added a Payment History screen.
- Modelled users, profiles, and insurance dependents (Django models, serializers, views, admin).
- Added Medical Records and Profile Settings with input **validation**; split the sign-up screen into components.
- Reworked routing so an emergency flows into a dashboard for hospital selection and booking history.

Phase 4 - The hospital dashboard, web side (Mar 2026)

- Built a separate **hospital-facing dashboard** as a **Next.js** web app.
- Started it on MongoDB, then **refactored it to use the Django backend** instead of talking to a database directly - one source of truth and one place that enforces the rules.
- Added a **proxy middleware** to handle authentication for the dashboard.
- Integrated **push notifications**.

Phase 5 - Production infrastructure and DevOps (May 2026)

- Added **Celery** for asynchronous background tasks (emails, push notifications), with a worker and a scheduler (**Celery beat**).
- Containerised everything with **Docker** and **Docker Compose**, with container **health checks** and entrypoint scripts.
- Put **Nginx** in front as a **reverse proxy**: HTTP/HTTPS, SSL certificates via **Certbot**, security headers, and gzip.
- **Cloudinary** for media storage; **Resend** for transactional email (OTP and welcome mails).

- Wired up observability: **Sentry** for error tracking, and a **Prometheus / Alertmanager / Loki / Grafana** monitoring stack with a metrics middleware.
- Built **CI/CD** (a **Jenkins** pipeline first, then **GitHub Actions**): build the Docker image, run checks, deploy, and **roll back** on failure.

Phase 6 - Engineering audit and the hard fixes (Jul 2026)

- Ran a formal **engineering audit** against an 18-part DevOps standard, with a written gap analysis.
- **Atomic bed allocation** so the last bed can never be oversold - the race-condition fix.
- A **bed-release state machine** so a bed is released exactly once.
- Fixed an **N+1 query** in hospital search and capped result counts.
- **JWT** hardening: short 15-minute access tokens, JWT-only API in production, tokens and PII kept in **SecureStore**.
- **Rate-limited** login, OTP, and password-reset endpoints.
- **Constant-time** signature comparison on payments (timing-attack safe).
- Fixed a **dashboard auth bypass** - the web dashboard was trusting self-asserted headers instead of a signed token (an **IDOR**-class access-control bug).
- Patched **21 HIGH/CRITICAL CVEs** flagged by the **Trivy** container scan.
- Wrote **regression tests** for bed concurrency, access control, and the N+1 guard.

Phase 7 - Deploy stabilisation and smart features (Jul 2026)

- Fixed real deployment failures: Nginx caching a dead backend IP (a 502), and production accidentally running host code instead of the built image.
- Built a **weighted recommendation engine** with an **escalation ladder** to match a patient to the right hospital.
- Drove hospital selection from the patient's **insurance policy** rather than a manual form.
- Added an **arrival-code** flow so the hospital confirms the patient at the desk (a staff-driven booking lifecycle).
- **Auto-expiry of stale reservations** - both on a schedule and lazily on read.
- Performance: virtualised the results list with **FlatList** and **memoised** the hospital card.

Part 2 - Questions and detailed answers

From Phase 1 (scaffolding)

"Why React Native and Expo instead of Flutter?"

React Native lets me build a real Android app using JavaScript, TypeScript, and React - the exact skills I already use for web. As a solo developer shipping fast, reusing my existing React knowledge mattered more than anything. Flutter is excellent and its UI can be smoother out of the box, but it uses Dart, which I would have had to learn from scratch, and I did not need that cost. Expo sits on top of React Native and handles the painful native tooling - builds, over-the-air updates, native modules - so one person can move quickly. The honest trade-off: Flutter might give slightly better UI polish; React Native gave me speed and skill reuse, which is what a one-person team needs.

"Why a monorepo?"

Keeping the mobile client and the backend in one repository meant one place to track changes, shared tooling, and simpler coordination when the API and the app change together. For a small team, that is less overhead than two separate repos that have to be kept in step.

From Phase 2 (authentication)

"Why Argon2 for password hashing?"

First, passwords are **hashed, not encrypted** - hashing is one-way, so even I can never read a user's password; I only ever compare hashes. Argon2 is a modern, memory-hard hashing algorithm that won the Password Hashing Competition. "Memory-hard" means it deliberately uses a lot of memory, which makes it expensive to crack on GPUs and custom hardware, where attackers get their speed. It is also salted - each password gets a unique random salt so two identical passwords hash differently, which defeats precomputed rainbow-table attacks. Compared to older options like bcrypt, Argon2 is the current recommended default.

"What is the difference between hashing and encryption?"

Encryption is two-way: you can encrypt and later decrypt with a key. Hashing is one-way: you cannot reverse it. Passwords should be hashed because I never need the original back - I only need to check whether a login attempt hashes to the same value. Payment signatures use HMAC, which is hashing with a secret key.

"Why Django over Node?"

Django is batteries-included - it ships an admin panel, an ORM, authentication scaffolding, and secure defaults. For a healthcare backend with a lot of business rules, built by one person, that structure and safety let me move faster and make fewer security mistakes. I do use Node and Express elsewhere when I want a lean, custom backend, but DokLink's breadth suited Django.

From Phase 3 (booking and payments)

"How does the Razorpay payment flow work?"

The app creates an order through my backend, the user pays through Razorpay, and Razorpay sends back a payment id and a **signature**. My backend recomputes that signature using **HMAC-SHA256** with my secret key and compares it - if they match, the payment is genuine and not forged. I manage the full lifecycle: created, paid, verified, failed. Later I made the comparison **constant-time** to avoid timing attacks, and I stopped sending the signature and payment id back to the client, because the client never needs them.

"How did you model the data?"

The core entities are users, hospitals, beds, bookings/reservations, and payments, plus profiles and insurance. Each is a table with a primary key; bookings link a user to a bed at a hospital through foreign keys; a payment belongs to a booking. It is a relational schema because these relationships and their consistency are the whole point of a booking system.

"What is a serializer / Django REST Framework?"

DRF is the layer that turns my Django models into a clean REST API for the app to call. A serializer converts between database objects and JSON, and validates incoming data - so a malformed request is rejected before it ever touches the database.

From Phase 4 (the dashboard)

"Why did you move the dashboard off MongoDB onto the Django backend?"

Originally the Next.js dashboard talked to its own MongoDB, which meant two sources of truth and a risk of the app and the dashboard disagreeing. I refactored it to go through the same Django backend the mobile app uses, so there is **one source of truth**, one place that enforces business rules and authentication, and no duplicated or drifting data. It is the same reasoning as choosing one well-structured system over a scattered one.

"Why Next.js for the dashboard?"

The dashboard is a web app for hospital staff, so server-side rendering, routing, and a solid React framework made sense. Next.js gave me that out of the box.

From Phase 5 (infrastructure)

"What is Celery and why did you need it?"

Celery is a task queue for doing work **asynchronously**, outside the request. When a user signs up, I do not want them waiting while an email is sent - so I hand that off to Celery and respond immediately; a background worker sends the email. It needs a **message broker** (I use Redis) to pass the task from the web process to the worker. **Celery beat** is the scheduler that runs periodic tasks - like expiring stale bed reservations on a timer.

"Walk me through your CI/CD pipeline."

On a push, the pipeline builds the Docker image, runs the linter and the test suite, and scans the image for vulnerabilities with Trivy. If everything passes, it deploys by shipping the new image and Compose files to the server and recreating the containers, then checks a health endpoint. If the health check fails, a **rollback** script restores the previous version. The point of CI/CD is that releases are repeatable and safe, not manual and scary.

"What do you monitor, and what is observability?"

Observability is being able to see what the system is doing from the outside. I use three kinds of signal: **metrics** (numbers over time, via Prometheus, shown in Grafana dashboards), **logs** (what happened, aggregated with Loki), and **error tracking / traces** (via Sentry, which tells me exactly where an exception happened). Alertmanager can page me when a metric crosses a threshold. The rule I follow: if I cannot see it failing, it is not really in production.

From Phase 6 (the audit and hard fixes)

"Explain the race condition and exactly how you fixed it." (your best answer)

A booking is really read-then-write: read the free-bed count, and if it is above zero, write the new count and create the reservation. The danger is two users booking the **last** bed at the same instant - both read "1 free" before either writes, so both proceed and the count goes to -1, meaning two people are promised one bed. I fixed it with an **atomic transaction** plus **row-level locking**: when a booking starts, the database locks that specific bed/hospital row, so the second request has to wait until the first finishes. By the time it is allowed to read, the count is already zero and it is correctly refused. Only one booking wins and the count can never go negative. I also added a **release state machine** so a bed is released exactly once, and **auto-expiry** so unused reservations return to the pool.

"What is row-level locking, and how is it different from table-level?"

A lock is the database temporarily reserving data so only one transaction can touch it. Row-level locking (in PostgreSQL, `SELECT ... FOR UPDATE` inside a transaction) locks only the specific row - so two people racing for the same bed are serialised safely, but bookings at other hospitals run in parallel. Table-level locking would freeze the whole table - correct, but it would serialise every booking everywhere, killing performance. Row-level is the balance of correctness and concurrency.

"What are the ACID properties?"

Atomicity (all steps happen or none do), Consistency (the database stays valid - a bed count never goes negative), Isolation (concurrent transactions do not interfere - this is what stopped the double-booking), and Durability (once committed, it survives a crash). My booking relies especially on Atomicity and Isolation.

"What is an N+1 query, and how did you fix it?"

An N+1 query is when you fetch a list of N things with one query, then accidentally run one more query per item to load its related data - so 1 query becomes 1+N. In the hospital search, loading each

hospital's related data separately blew up the query count. I fixed it by fetching the related data up front in a single efficient query (in Django, `select_related` for foreign keys and `prefetch_related` for reverse/many relations), and I capped the number of results returned. I even added a regression test so the N+1 can never quietly come back.

"Why 15-minute JWT access tokens?"

A JWT is a signed token the server issues at login; the client sends it with each request and the server verifies the signature instead of storing a session. I keep the **access token** short-lived - 15 minutes - so that if one is ever stolen, it is useless almost immediately. A longer-lived **refresh token** is used to quietly get new access tokens, and I store both in **SecureStore** (the phone's encrypted keychain/keystore), not in plain storage.

"What is rate limiting protecting against?"

Capping how many requests an IP or account can make in a time window. On login, OTP, and password-reset I rate-limit to stop brute-force and credential-stuffing attacks - someone trying thousands of passwords or spamming OTPs. Without it, those endpoints are wide open to automated abuse.

"Why constant-time signature comparison?"

If you compare two signatures with a normal comparison that stops at the first differing byte, the time it takes leaks how many leading bytes matched - a **timing attack** that lets an attacker guess a valid signature byte by byte. A constant-time comparison always takes the same time regardless of where they differ, so nothing leaks. I used the language's built-in constant-time compare for the payment signature.

"What is an IDOR / the auth bypass you fixed?"

IDOR - Insecure Direct Object Reference - is when you can access someone else's data just by changing an id, because the server never checks you are allowed to. My dashboard had a related bug: the web app was authenticating with **self-asserted headers** - effectively telling the backend who it was - instead of proving it with a **signed token**. I fixed it by requiring real token authentication on the dashboard API, so identity is verified, not claimed.

"What is a CVE and how do you scan for them?"

A CVE - Common Vulnerabilities and Exposures - is a public identifier for a known security flaw in a piece of software. **Trivy** scans my Docker image and its dependencies against the CVE databases. It flagged 21 HIGH/CRITICAL issues, which I fixed by upgrading the affected libraries and removing build-time tooling from the runtime image, so the shipped container has a much smaller attack surface.

From Phase 7 (stabilisation and features)

"How does your recommendation engine work?"

It scores hospitals with a **weighted** combination of factors - distance, bed availability, care level needed, and the patient's insurance policy - and uses an **escalation ladder**: if the required level of care is not

available nearby, it escalates to the next appropriate option rather than failing. It replaced a manual form, so the patient is guided to the right hospital instead of guessing.

"How do stale reservations get cleaned up?"

Two ways, deliberately. A scheduled job (Celery beat) expires reservations whose window has passed and returns their beds to the pool. And lazily on read - when the system looks at bed availability, any expired reservations are released then and there. Belt and suspenders, so a stuck reservation can never hold a bed hostage.

"You had a 502 in production - what happened?"

Nginx had cached the backend's IP address, and when the backend container got a new IP on redeploy, Nginx kept sending traffic to the dead one, returning 502s. I fixed the deploy so Nginx is recreated (not just reloaded) and always resolves the current backend. It taught me that a lot of production failures are not in the code - they are in how the pieces are wired together.

"How do you keep a long list fast in React Native?"

I used **FlatList**, which is virtualised - it only renders the rows currently on screen and recycles them, instead of rendering hundreds at once like a plain ScrollView. I also **memoised** the hospital card component so unchanged cards do not re-render. Together that keeps scrolling smooth even on low-end phones, which is exactly where emergencies happen.

Part 3 - The bait-terms dictionary

Quick, detailed reference for every term you might drop. Each has the answer and a short "say it like this."

React Native

A framework for building real mobile apps using JavaScript and React. *Say it like this:* "I build the app in React Native, so I reuse my React skills across web and mobile."

Expo

A toolkit on top of React Native that handles builds, over-the-air updates, and native modules. *Say it:* "Expo removes the native-tooling pain so a solo developer can ship fast."

Argon2

A modern, memory-hard, salted password-hashing algorithm; harder to crack on GPUs than bcrypt. *Say it:* "Passwords are hashed with Argon2 - one-way and memory-hard, so they can't be reversed or brute-forced cheaply."

Hashing vs encryption

Hashing is one-way (passwords, signatures); encryption is two-way with a key (data in transit). *Say it:* "Passwords are hashed, not encrypted - I never need the original back."

Django / DRF / serializer / ORM

Django is a batteries-included Python web framework; DRF exposes it as a REST API; a serializer converts and validates data between the database and JSON; the ORM lets you use the database through code objects. *Say it:* "DRF gives me a clean REST API, and serializers validate every request before it hits the DB."

JWT (access vs refresh)

A signed token proving identity, sent with each request; the server verifies the signature (stateless). Short access tokens (15 min) limit damage if stolen; a refresh token gets new ones. *Say it:* "15-minute JWTs so a stolen token dies fast, refreshed with a longer-lived refresh token."

SecureStore

The phone's encrypted storage (iOS Keychain / Android Keystore) for secrets like tokens - not plain AsyncStorage. *Say it:* "Tokens and PII live in SecureStore, encrypted by the OS, never in plain storage."

Race condition

When two operations on shared data at the same time give a wrong result depending on timing. *Say it:* "Two users booking the last bed at once - a classic race condition."

Atomic transaction

A group of database operations that all succeed or all roll back, as one unit. *Say it:* "The booking runs in an atomic transaction, so it's all-or-nothing."

Row-level locking

Locking just the specific row being changed (`SELECT ... FOR UPDATE`), so only contenders for that exact row wait. *Say it:* "Row-level locking serialises people racing for the same bed but lets other bookings run in parallel."

ACID

Atomicity, Consistency, Isolation, Durability - the four guarantees of a transaction. *Say it:* "I lean on Atomicity and Isolation for the bed booking."

State machine (bed release)

Modelling a reservation's status as fixed states with guarded transitions, so a bed is released exactly once. *Say it:* "A release state machine makes bed release idempotent - it can't double-release."

Auto-expiry (scheduled + lazy)

Freeing stale reservations both on a timer and when data is read. *Say it:* "Stale reservations expire on a schedule and lazily on read, so a bed can't get stuck."

N+1 query

One query for a list plus one extra per item = 1+N; fixed by loading related data up front (`select_related` / `prefetch_related`). *Say it:* "I killed an N+1 in hospital search by prefetching related data and capping results."

Rate limiting

Capping requests per IP/account/time to stop brute force and abuse. *Say it:* "Login, OTP, and password-reset are rate-limited against brute force."

HMAC-SHA256

A signature made from data plus a secret key using the SHA-256 hash; proves a message is genuine and untampered. *Say it:* "I verify Razorpay's payment signature with HMAC-SHA256 against my secret."

Constant-time comparison

Comparing secrets in fixed time so response timing can't leak how many bytes matched (defeats timing attacks). *Say it:* "The signature check is constant-time, so it can't be guessed byte by byte."

IDOR / access control

Accessing another user's object by changing an id because authorization isn't checked; fixed by verifying identity. *Say it:* "I closed a dashboard auth bypass - it trusted self-asserted headers instead of a signed token."

CVE / Trivy

A CVE is a public id for a known vulnerability; Trivy scans container images for them. *Say it:* "Trivy flagged 21 HIGH/CRITICAL CVEs; I patched them by upgrading deps and slimming the runtime image."

Reverse proxy / Nginx

Nginx sits in front, receives all traffic, forwards it to the backend, and handles HTTPS, headers, and static files. *Say it:* "Nginx is my reverse proxy - it terminates TLS and forwards to the backend."

TLS / SSL / Certbot / HTTPS

HTTPS is HTTP encrypted with TLS; Certbot gets and auto-renews free Let's Encrypt certificates. *Say it:* "SSL is automated with Certbot, so HTTPS certs renew themselves."

Security headers

HTTP headers (HSTS, X-Frame-Options, X-Content-Type-Options, CSP) that harden the browser against common attacks. *Say it:* "Nginx sets strict security headers like HSTS."

Celery / message broker / Celery beat

Celery runs background tasks off the request; a broker (Redis) passes tasks to workers; beat schedules periodic ones. *Say it:* "Emails go through Celery so the request doesn't block; beat runs the scheduled jobs."

Docker / Docker Compose / container vs VM

Docker packages the app into a container that runs the same everywhere; Compose orchestrates several containers; a container shares the host kernel and is lighter than a VM. *Say it:* "Everything runs in Docker, orchestrated with Compose - backend, Nginx, Celery, Redis."

CI/CD / rollback

Automated build, test, scan, and deploy on every push, with a rollback if the health check fails. *Say it:* "CI/CD builds, tests, scans, and deploys, and rolls back automatically on a failed health check."

Observability (metrics, logs, traces)

Metrics (Prometheus/Grafana), logs (Loki), errors/traces (Sentry), alerts (Alertmanager). *Say it:* "Metrics in Prometheus and Grafana, logs in Loki, errors in Sentry - if I can't see it fail, it doesn't ship."

Cloudinary

A managed service for storing and optimising media instead of putting files on my server. *Say it:* "Media goes to Cloudinary, optimised and off my own disk."

Push notifications

Server-initiated messages to the app even when it is closed. *Say it:* "Push notifications keep users updated on their booking status."

FlatList / memoisation

FlatList only renders visible rows (virtualisation); React.memo stops unchanged components re-rendering. *Say it:* "FlatList virtualises the results and I memoise the cards, so long lists stay smooth on cheap phones."

Next.js

A React framework with server-side rendering and routing; I used it for the hospital dashboard. *Say it:* "The staff dashboard is a Next.js app talking to the same Django backend."

Monolith vs microservices

One well-structured codebase vs many small networked services; I chose the monolith for a small team. *Say it:* "A single Django monolith - one codebase a solo developer can actually operate, not a distributed system to babysit."

Recommendation engine / escalation ladder

Weighted scoring of hospitals with a fallback ladder when the needed care level isn't available nearby. *Say it:* "A weighted engine ranks hospitals and escalates up a ladder if the right level isn't nearby."

PostgreSQL vs MongoDB

PostgreSQL is relational with transactions and integrity; MongoDB is flexible documents. Bookings need transactions, so PostgreSQL. *Say it:* "PostgreSQL, because a booking system lives on transactions and integrity, not schema flexibility."

Part 4 - Sample code snippets (pen and paper)

They may hand you paper and ask you to sketch a piece of this. These are short, correct, and defensible - narrate the one-line logic as you write, exactly as the seniors said. You do not need them byte-perfect; the logic is what counts.

The atomic bed booking (the race-condition fix) - Django

```
from django.db import transaction

def book_bed(user, hospital_id, bed_type):
    with transaction.atomic():              # all-or-nothing
        bed = (Bed.objects
               .select_for_update()         # row-level lock
               .filter(hospital_id=hospital_id,
                       type=bed_type,
                       status="FREE")
               .first())
        if bed is None:
            raise NoBedAvailable()           # the other user won
        bed.status = "RESERVED"
        bed.save()
        return Reservation.objects.create(
            user=user, bed=bed,
            expires_at=now() + travel_window(user, hospital_id),
        )
```

Say aloud: "`select_for_update` locks that bed row inside the transaction, so a second booking waits, then finds no FREE bed and is refused. The count can't go negative."

Razorpay signature verification (constant-time)

```
import hmac, hashlib

def verify_payment(order_id, payment_id, signature, secret):
    body = f"{order_id}|{payment_id}".encode()
    expected = hmac.new(secret.encode(), body,
                        hashlib.sha256).hexdigest()
    return hmac.compare_digest(expected, signature)  # constant-time
```

Say aloud: "I recompute the HMAC-SHA256 with my secret and compare in constant time, so a forged signature is rejected and timing can't leak the answer."

Killing the N+1 query

```python
# Bad: 1 query for hospitals, then 1 more per hospital for its beds
hospitals = Hospital.objects.all()

# Good: fetch the related rows up front, and cap the result
hospitals = (Hospital.objects
             .select_related("city")        # FK  -> SQL JOIN
             .prefetch_related("beds")[:20]) # reverse FK, capped
```

Say aloud: "`select_related` joins the one-to-one/foreign-key data; `prefetch_related` batches the reverse relations - so it's a couple of queries, not 1+N."

Rate limiting a sensitive endpoint (DRF)

```python
from rest_framework.throttling import AnonRateThrottle

class LoginThrottle(AnonRateThrottle):
    rate = "5/min"      # 5 attempts per IP per minute
```

Say aloud: "Login, OTP, and password-reset get a throttle so brute force is capped per IP."

Store the JWT safely on the phone (React Native)

```javascript
import * as SecureStore from "expo-secure-store";

await SecureStore.setItemAsync("access", token);  // OS keychain/keystore
const token = await SecureStore.getItemAsync("access");
```

Say aloud: "Tokens go in SecureStore, encrypted by the OS keystore, never in plain AsyncStorage."

A DRF serializer (validation before the DB)

```python
class ReservationSerializer(serializers.ModelSerializer):
    class Meta:
        model = Reservation
        fields = ["id", "hospital", "bed_type", "expires_at"]
        read_only_fields = ["id", "expires_at"]
```

Say aloud: "The serializer validates and shapes the request before it ever reaches the database."

The 12 hooks to drop on purpose

If you want to steer them onto your strongest ground, plant these and pause:

1. Race condition / atomic bed allocation
2. Row-level locking
3. N+1 query
4. 15-minute JWTs
5. Constant-time signature check
6. Rate limiting
7. Reverse proxy
8. Monolith over microservices
9. React Native over Flutter
10. Moved the dashboard off MongoDB onto one backend
11. Auto-expiry via scheduled + lazy release
12. Trivy / CVE patching

Every one of these has a full answer above. Say the term, stop talking, and let them ask.